# ITCS213 Computer Organization and Architecture

Class: 1-2026
Sec 2 Fri 9:00-12:00 IT312 (Aj. Joke)
Sec 3 Fri 13:00-16:00 IT312 (Aj. Joke)
Sec 1 Fri 13:00-16:00 IT311 (Aj. Pete)

Unit 6: Signed Integer, Logical Operations

**Lecturer Dr. Kanoksak Wattanachote (Ph.D) (金功勳 博士)**
**kanoksak.wat@mahidol.ac.th**

# Grading

## ITCS213 Computer Organization and Architecture

- Overall Score Parts
  - Class activities + Assignments + Quizzes 20%
  - Midterm Exam 40%
  - Final Exam 40%

**GRADING SYSTEM**

| | | | |
|---|---|---|---|
| Grade Criteria | A | >= | 80 |
| | B+ | >= | 75 |
| | B | >= | 70 |
| | C+ | >= | 65 |
| | C | >= | 60 |
| | D+ | >= | 55 |
| | D | >= | 50 |
| | F | < | 50 |

# Content

## Unit 6: Signed Integer, Logical Operations

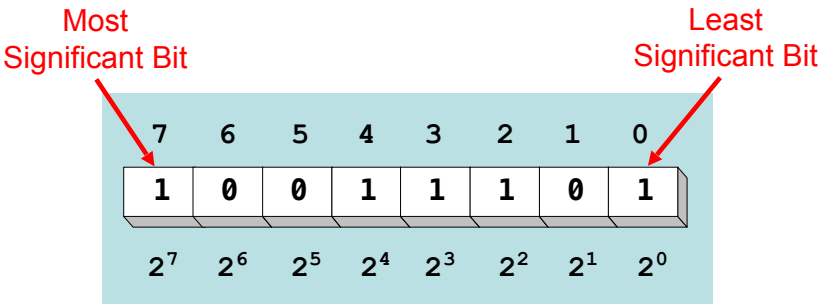
- Positional Number System and Integer
- Write Through and Write Back
- MIPS General-Purpose Registers
- R-Type Instruction Format
- R-Type Arithmetic, Logical, and Shift Instructions
- I-Type Instruction Format and Immediate Constants
- I-type Arithmetic and Logical Instructions
- Pseudo Instructions

# Positional Number System and Integer

# Binary Numbers

## Unit 6: Signed Integer, Logical Operations

- Each binary digit (called a bit) is either 1 or 0
- Bits have no inherent meaning and can represent
  - Unsigned and signed integers
  - Characters
  - Floating-point numbers
  - Images, sound, etc.
- Bit Numbering
  - Least significant bit (LSB) is rightmost (bit 0)
  - Most significant bit (MSB) is leftmost (bit 7 in an 8-bit number)



# Converting Unsigned Decimal to Binary

## Unit 6: Signed Integer, Logical Operations

- To convert a decimal integer to binary, repeatedly divide the decimal integer by 2.
- Each remainder is a binary digit in the translated value.

| Division | Quotient | Remainder |
|---|---|---|
| 37/2 | 18 | 1 |
| 18/2 | 9 | 0 |
| 9/2 | 4 | 1 |
| 4/2 | 2 | 0 |
| 2/2 | 1 | 0 |
| 1/2 | 0 | 1 |

Least Significant Bit

Most Significant Bit

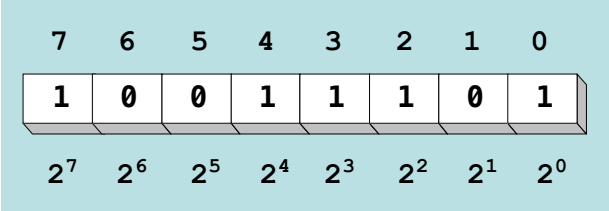
Stop when the Quotient is zero

$37 = (100101)_2$

# Converting Binary to Decimal

## Unit 6: Signed Integer, Logical Operations

- Each bit represents a power of 2.
- Every binary number is a sum of powers of 2.
- Decimal Value = $(d_{n-1} \times 2^{n-1}) + \ldots + (d_1 \times 2^1) + (d_0 \times 2^0)$
  - Binary $(10011101)_2 = 2^7 + 2^4 + 2^3 + 2^2 + 2^0$
  - Binary $(10011101)_2 = 128 + 16 + 8 + 4 + 1$
  - Binary $(10011101)_2 = 157$




Some common powers of 2

| $2^n$ | Decimal | $2^n$ | Decimal |
|---|---|---|---|
| $2^0$ | 1 | $2^8$ | 256 |
| $2^1$ | 2 | $2^9$ | 512 |
| $2^2$ | 4 | $2^{10}$ | 1024 |
| $2^3$ | 8 | $2^{11}$ | 2048 |
| $2^4$ | 16 | $2^{12}$ | 4096 |
| $2^5$ | 32 | $2^{13}$ | 8192 |
| $2^6$ | 64 | $2^{14}$ | 16384 |
| $2^7$ | 128 | $2^{15}$ | 32768 |

# Hexadecimal Integers

## Unit 6: Signed Integer, Logical Operations

- 16 / Hexadecimal Digits: `0 – 9`, `A – F`.
- More convenient to use than binary numbers.

Binary, Decimal, and Hexadecimal Equivalents

| Binary | Decimal | Hexadecimal | Binary | Decimal | Hexadecimal |
|---|---|---|---|---|---|
| 0000 | 0 | 0 | 1000 | 8 | 8 |
| 0001 | 1 | 1 | 1001 | 9 | 9 |
| 0010 | 2 | 2 | 1010 | 10 | A |
| 0011 | 3 | 3 | 1011 | 11 | B |
| 0100 | 4 | 4 | 1100 | 12 | C |
| 0101 | 5 | 5 | 1101 | 13 | D |
| 0110 | 6 | 6 | 1110 | 14 | E |
| 0111 | 7 | 7 | 1111 | 15 | F |

# Converting Binary to Hexadecimal

## Unit 6: Signed Integer, Logical Operations

- Each hexadecimal digit corresponds to 4 binary bits.

- Example:
  - Convert the 32-bit binary number to its corresponding hexadecimal representation.

```
1110 1011 0001 0110 1010 0111 1001 0100
```

- Solution:

| E | B | 1 | 6 | A | 7 | 9 | 4 |
|---|---|---|---|---|---|---|---|
| 1110 | 1011 | 0001 | 0110 | 1010 | 0111 | 1001 | 0100 |

# Converting Hexadecimal to Decimal

## Unit 6: Signed Integer, Logical Operations

- Multiply each digit by its corresponding power of 16.

  Decimal Value = $(d_{n-1} \times 16^{n-1}) + (d_{n-2} \times 16^{n-2}) + \ldots + (d_1 \times 16^1) + d_0$

- Examples:

  $(1234)_{16} = (1 \times 16^3) + (2 \times 16^2) + (3 \times 16^1) + 4$

  $(1234)_{16} = 4660$

  $(3BA4)_{16} = (3 \times 16^3) + (11 \times 16^2) + (10 \times 16^1) + 4$

  $(3BA4)_{16} = 15268$

# Converting Decimal to Hexadecimal

## Unit 6: Signed Integer, Logical Operations

- To convert a decimal integer to hexadecimal, repeatedly divide the decimal integer by 16.
- Each remainder is a hex digit in the translated value.

| Division | Quotient | Remainder |
|---|---|---|
| 422/16 | 26 | 6 |
| 26/16 | 1 | A |
| 1/6 | 0 | 1 |

Least Significant Digit

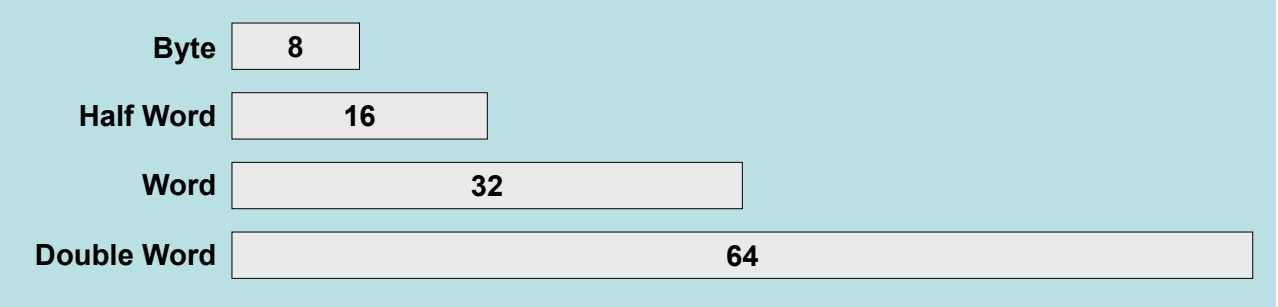
Most Significant Digit

Stop when the Quotient is zero

$$422 = (1A6)_{16}$$

# Integer Storage Sizes

## Unit 6: Signed Integer, Logical Operations



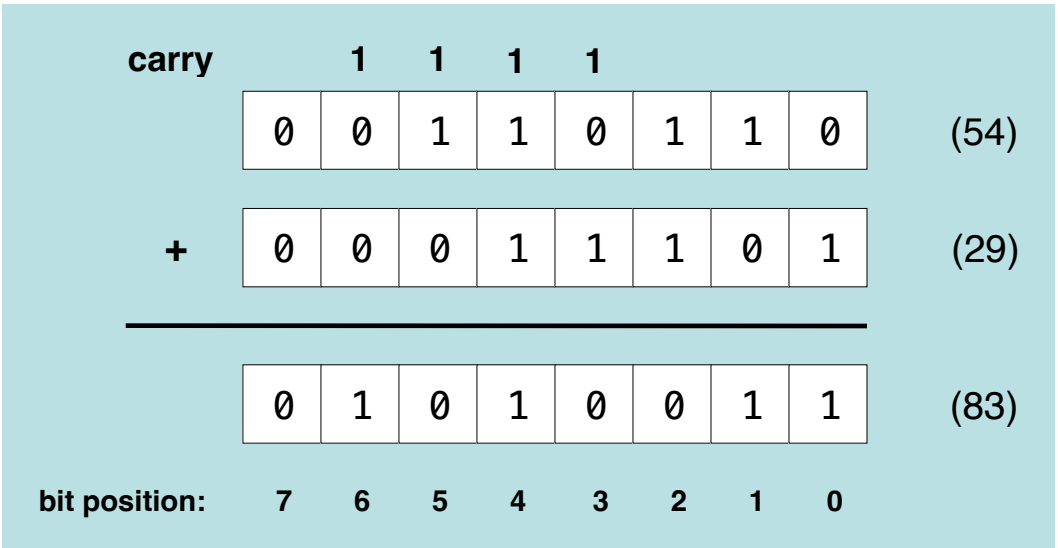

| Storage Type | Unsigned Range | Powers of 2 |
|---|---|---|
| **Byte** | `0 to 255` | `0 to` $(2^8 - 1)$ |
| **Half Word** | `0 to 65,535` | `0 to` $(2^{16} - 1)$ |
| **Word** | `0 to 4,294,967,295` | `0 to` $(2^{32} - 1)$ |
| **Double Word** | `0 to 18,446,744,073,709,551,615` | `0 to` $(2^{64} - 1)$ |

- What is the largest `20`-bit unsigned integer?
- Answer:

# Binary Addition

## Unit 6: Signed Integer, Logical Operations

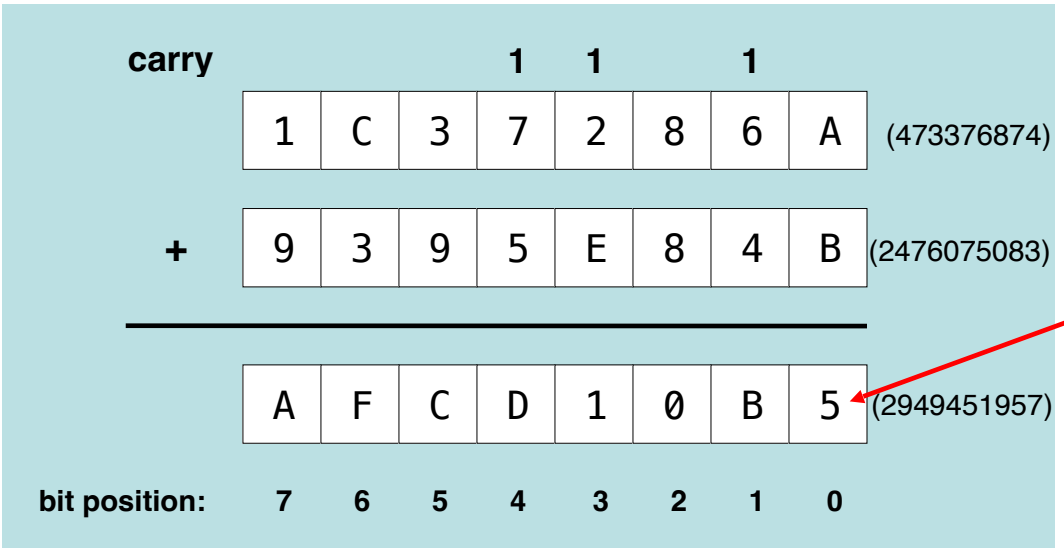
- Start with the least significant bit (rightmost bit).
- Add each pair of bits.
- Include the carry in the addition, if present.



# Hexadecimal Addition

## Unit 6: Signed Integer, Logical Operations

- Start with the least significant hexadecimal digits.
- Let Sum = the sum of two hex digits.
- If the Sum is greater than or equal to 16.
    - Sum = Sum – 16 and Carry = 1




A + B = 10 + 11 = 21
Since 21 ≥ 16
Sum = 21 – 16 = 5
Carry = 1

# Signed Integers

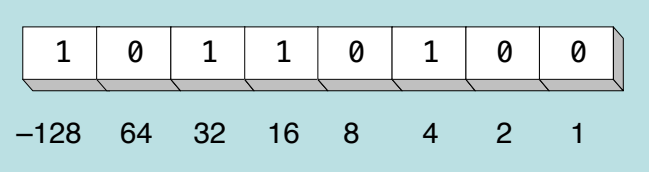
## Unit 6: Signed Integer, Logical Operations

- Several ways to represent a signed number:
  - Sign-Magnitude
  - Biased
  - 1's complement
  - 2's complement
- Divide the range of values into 2 equal parts
  - First part corresponds to the positive numbers (≥ 0).
  - Second part corresponds to the negative numbers (< 0).
- Focus will be on the 2's complement representation
  - Has many advantages over other representations.
  - Used widely in processors to represent signed integers.

# Two's Complement Representation

## Unit 6: Signed Integer, Logical Operations

- Positive numbers
  - Signed value = Unsigned value
- Negative numbers
  - Signed value = Unsigned value – $2^n$, $n$ = number of bits.
- Negative weight for MSB
  - Another way to obtain the signed value is to assign a negative weight to the most-significant bit.




= –128 + 32 + 16 + 4 = –76

Most Significant bit/digit

Most Significant bit/digit

| 8-bit Binary value | Unsigned value | Signed value |
|---|---|---|
| 00000000 | 0 | 0 |
| 00000001 | 1 | +1 |
| 00000010 | 2 | +2 |
| . . . | . . . | . . . |
| 01111110 | 126 | +126 |
| 01111111 | 127 | +127 |
| 10000000 | 128 | -128 |
| 10000001 | 129 | -127 |
| . . . | . . . | . . . |
| 11111110 | 254 | -2 |
| 11111111 | 255 | -1 |

# Forming the Two's Complement

## Unit 6: Signed Integer, Logical Operations



| | |
|---|---|
| starting value | `00100100 = +36` |
| step1: reverse the bits (1's complement) | `11011011` |
| step 2: add 1 to the value from step 1 | `+ 1` |
| sum = 2's complement representation | `11011100 = -36` |

- Sum of an integer and its 2's complement must be zero:
  `00100100 + 11011100` = `00000000` (8-bit sum) ⇒ Ignore Carry

- Another way to obtain the 2's complement:
  - Start at the least significant 1
  - Leave all the 0s to its right unchanged
  - Complement all the bits to its left

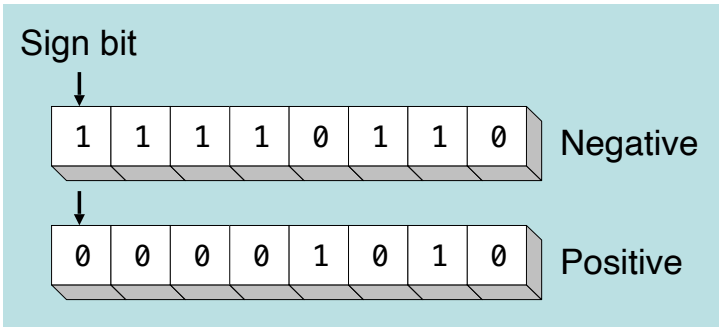
```
Binary Value          least
= 00100 1 00          significant 1
2's Complement
= 11011 1 00
```



# Sign Bit

## Unit 6: Signed Integer, Logical Operations

- The highest bit indicates the sign:
  - 1 = negative
  - 0 = positive

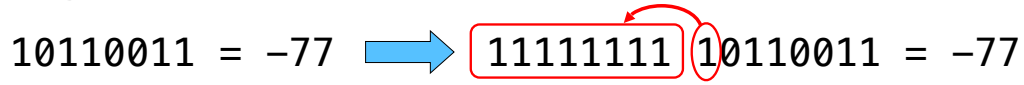



- For Hexadecimal Numbers, check the most significant digit:
  - If the highest digit is > 7, then the value is negative
  - Examples:
    - 8A and C5 are negative bytes
    - B1C42A00 is a negative word (32-bit signed integer)

# Sign Extension

## Unit 6: Signed Integer, Logical Operations

- Step 1: Move the number into the lower-significant bits
- Step 2: Fill all the remaining higher bits with the sign bit
  - This will ensure that both magnitude and sign are correct
  - Examples
    - Sign-Extend `10110011` to 16 bits:



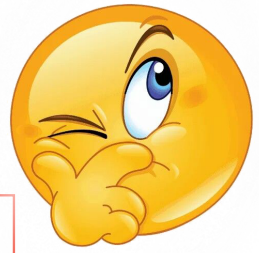

    - Sign-Extend `01100010` to 16 bits:




- Infinite 0s can be added to the left of a positive number
- Infinite 1s can be added to the left of a negative number

# Two's Complement of a Hexadecimal

## Unit 6: Signed Integer, Logical Operations



- To form the two's complement of a hexadecimal:
  - Subtract each hexadecimal digit from 15
  - Add 1

Inverse of
6 —> 9
A —> 5
3 —> C
D —> 2

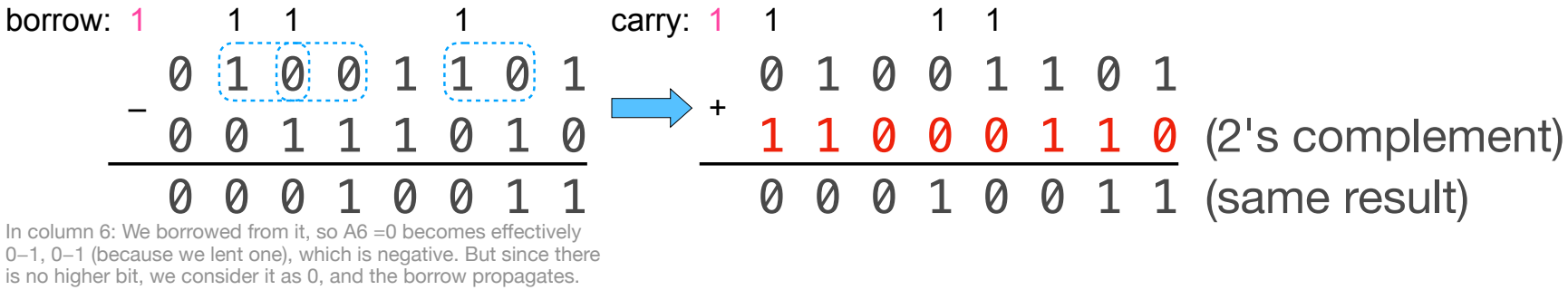
- Examples:
  - 2's complement of 6A3D = 95C2 + 1 = 95C3
  - 2's complement of 92F15AC0 = 6D0EA53F + 1 = 6D0EA540
  - 2's complement of FFFFFFFF = 00000000 + 1 = 00000001

- No need to convert hexadecimal to binary

# Binary Subtraction

## Unit 6: Signed Integer, Logical Operations

- When subtracting A – B, convert B to its 2's complement.
- Add A to (–B)



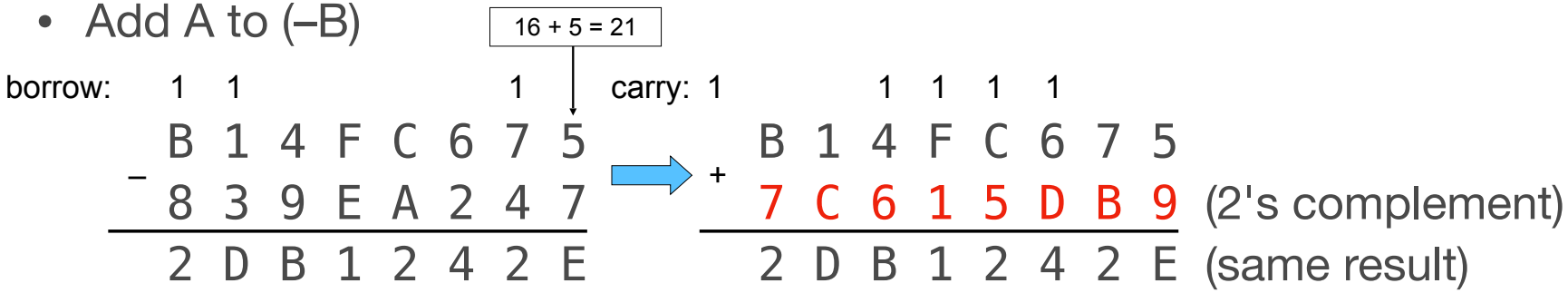

In column 6: We borrowed from it, so A6 =0 becomes effectively 0–1, 0–1 (because we lent one), which is negative. But since there is no higher bit, we consider it as 0, and the borrow propagates.

- Final carry is ignored because
  - Negative number is sign-extended with 1's.
  - You can imagine infinite 1's to the left of a negative number.
  - Adding the carry to the extended 1's produces extended zeros.

# Hexadecimal Subtraction

## Unit 6: Signed Integer, Logical Operations

- When subtracting A – B, convert B to its 2's complement.
- Add A to (–B)




- When a borrow is required from the digit to the left, then
  - Add 16 (decimal) to the current digit's value
  - Last carry is ignored

# Range of Signed Integers

## Unit 6: Signed Integer, Logical Operations






- For n-bit signed integers: Range is $-2^{n-1}$ to $(2^{n-1} - 1)$
- Positive range: $0$ to $2^{n-1} - 1$
- Negative range: $-2^{n-1}$ to $-1$

| Storage Type | Signed Range | Powers of 2 |
|---|---|---|
| **Byte** | –128 to 127 | $-2^7$ to $(2^7 - 1)$ |
| **Half Word** | –32,768 to 32,767 | $-2^{15}$ to $(2^{15} - 1)$ |
| **Word** | –2,147,483,648 to 2,147,483,647 | $-2^{31}$ to $(2^{31} - 1)$ |
| **Double Word** | –9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 | $-2^{63}$ to $(2^{63} - 1)$ |

- Practice: What is the range of signed values that may be stored in 26 bits?
- Answer:

# Carry and Overflow

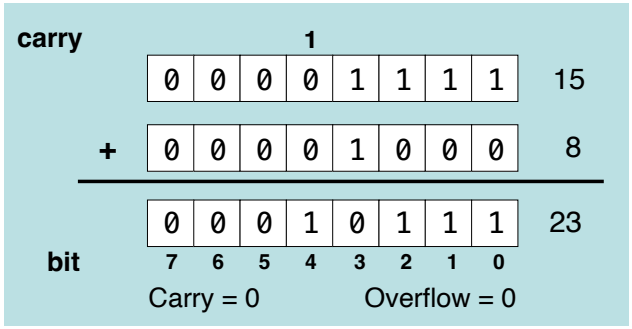
## Unit 6: Signed Integer, Logical Operations

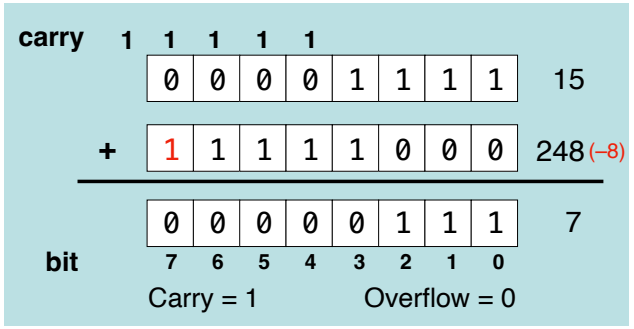
- Carry is important when …
  - Adding or subtracting unsigned integers.
  - Indicates that the unsigned sum is out of range.
  - Either < 0 or > the maximum unsigned n-bit value.
- Overflow is important when …
  - Adding or subtracting signed integers.
  - Indicates that the signed sum is out of range.
  - Overflow occurs when …
    - Adding two positive numbers and the sum is negative.
    - Adding two negative numbers and the sum is positive.
    - It can happen because of the fixed number of sum bits.

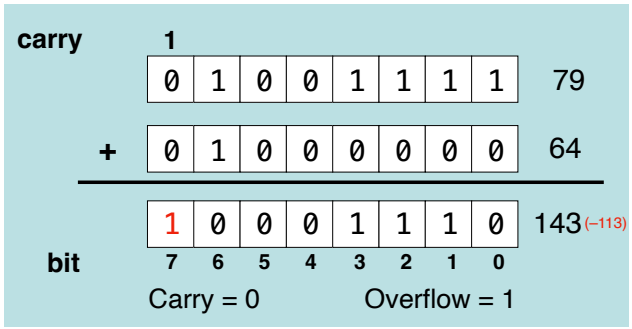
# Carry and Overflow Examples

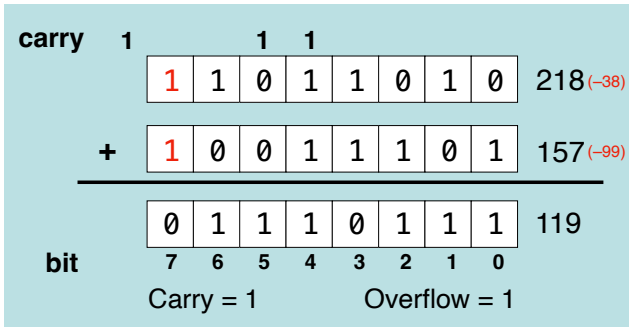
## Unit 6: Signed Integer, Logical Operations

- We can have carry without overflow and vice versa.
- Four cases are possible (Examples are 8-bit numbers).



# Character Storage

## Unit 6: Signed Integer, Logical Operations

- Character sets
  - Standard ASCII: 7-bit character codes (0 – 127).
  - Extended ASCII: 8-bit character codes (0 – 255).
  - Unicode: 16-bit character codes (0 – 65,535).
  - Unicode standard represents a universal character set:
    - Defines codes for characters used in all major languages.
    - Used in Windows XP: each character is encoded as 16 bits.
  - UTF-8: variable-length encoding used in HTML:
    - Encodes all Unicode characters.
    - Uses 1 byte for ASCII, but multiple bytes for other characters.
- Null-terminated String
  - Array of characters followed by a NULL character.

# Printable ASCII Codes

## Unit 6: Signed Integer, Logical Operations

- Examples:
  - ASCII code for the space character = 20 (hex) = 32 (decimal).
  - ASCII code for 'L' = 4C (hex) = 76 (decimal).
  - ASCII code for 'a' = 61 (hex) = 97 (decimal).

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | A | B | C | D | E | F |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 2 | space | ! | " | # | $ | % | & | ' | ( | ) | * | + | , | - | . | / |
| 3 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | : | ; | < | = | > | ? |
| 4 | @ | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O |
| 5 | P | Q | R | S | T | U | V | W | X | Y | Z | [ | \ | ] | ^ | _ |
| 6 | ` | a | b | c | d | e | f | g | h | i | j | k | l | m | n | o |
| 7 | p | q | r | s | t | u | v | w | x | y | z | { | \| | } | ~ | DEL |

# Control Characters

## Unit 6: Signed Integer, Logical Operations

- The first 32 characters of the ASCII table are used for control.
- Control character codes = 00 to 1F (hexadecimal)
    - Not shown in the previous slide.
- Examples of Control Characters:
    - Character 0 is the NULL character ⇒ used to terminate a string.
    - Character 9 is the Horizontal Tab (HT) character.
    - Character 0A (hex) = 10 (decimal) is the Line Feed (LF).
    - Character 0D (hex) = 13 (decimal) is the Carriage Return (CR).
    - The LF and CR characters are used together.
        - They advance the cursor to the beginning of the next line.
- One control character appears at the end of the ASCII table.
    - Character 7F (hex) is the Delete (DEL) character.

# Write Through & Write back Organizations

# Cache

## Unit 5: MIPS Assembly, Cache, Memory, and Integer

- Cache write policies
  - When the CPU performs a store operation, the cache handles the update according to its write policy:
    - Write-through: The data is written to both the **cache** and **main memory** simultaneously.
      - This is simpler to implement and maintains consistent memory, but it is slower for writes.
    - Write-back: The data is only written to the **cache** initially.
      - The cache block is marked "**dirty**."
      - The updated data is written back to main memory **only** when that cache block is evicted to make room for new data.
      - This offers better write performance but is more complex.

# Write Efficiency

## Unit 6: Signed Integer, Logical Operations

| Feature | Write-Through | Write-Back |
|---|---|---|
| Complexity | Simpler. No dirty bits, simple eviction. | More Complex. Requires dirty bits and write-back on eviction. |
| Memory Consistency | Excellent. Cache and memory are always consistent. | Poor. Memory can be stale until a dirty line is evicted. |
| Write Speed | Slow. Every write goes to slow DRAM. | Fast. Writes happen at cache speed; memory update is delayed. |
| Memory Bus Traffic | High. Every write creates bus traffic. | Low. Only dirty evictions create traffic; writes are absorbed. |

# Integer ADD/SUB Operations (Recap)

# R-Type Integer Add and Subtract

## Unit 6: Signed Integer, Logical Operations

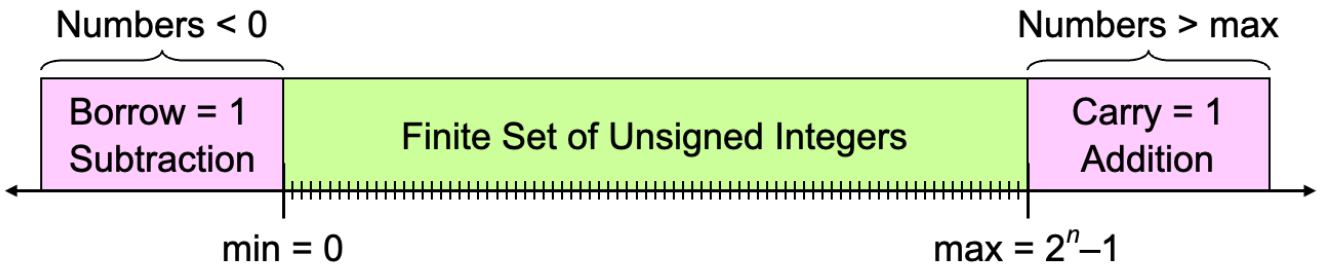
| Instruction | Meaning | Op | Rs | Rt | Rd | sa | func |
|---|---|---|---|---|---|---|---|
| `add $t1, $t2, $t3` | `$t1 = $t2 + $t3` | 0 | $t2 | $t3 | $t1 | 0 | 0x20 |
| `addu $t1, $t2, $t3` | `$t1 = $t2 + $t3` | 0 | $t2 | $t3 | $t1 | 0 | 0x21 |
| `sub $t1, $t2, $t3` | `$t1 = $t2 – $t3` | 0 | $t2 | $t3 | $t1 | 0 | 0x22 |
| `subu $t1, $t2, $t3` | `$t1 = $t2 – $t3` | 0 | $t2 | $t3 | $t1 | 0 | 0x23 |

- add, sub: arithmetic overflow causes an exception
  - In case of overflow, the result is not written to the destination register.
- addu, subu: arithmetic overflow is ignored
- addu, subu: compute the same result as add, sub
- Many programming languages ignore overflow
  - The + operator is translated into addu
  - The – operator is translated into subu

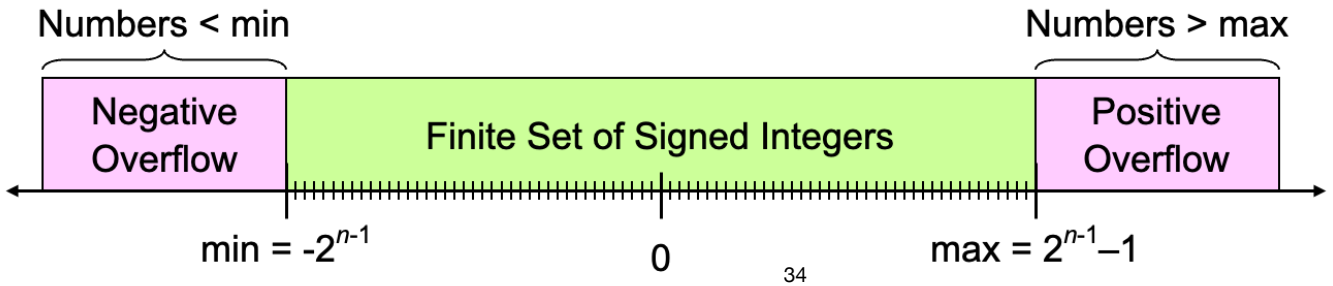
# Range, Carry, Borrow, and Overflow

## Unit 6: Signed Integer, Logical Operations

- Bits have NO meaning?
- The same n bits stored in a register can represent an unsigned or a signed integer.
- Unsigned Integers: n-bit representation




- Signed Integers: n-bit 2's complement representation

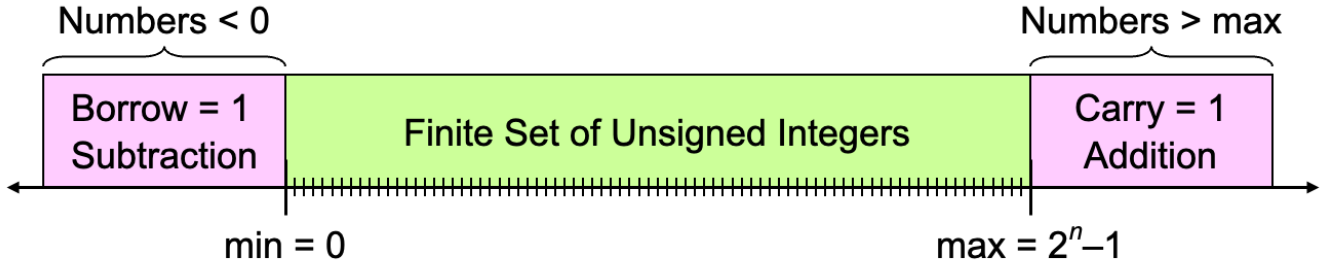


# Integer Unsigned ADD/SUB Operations

# Range, Carry, Borrow, and Overflow

## Unit 6: Signed Integer, Logical Operations

- n-bit Unsigned Integers: n-bit representation

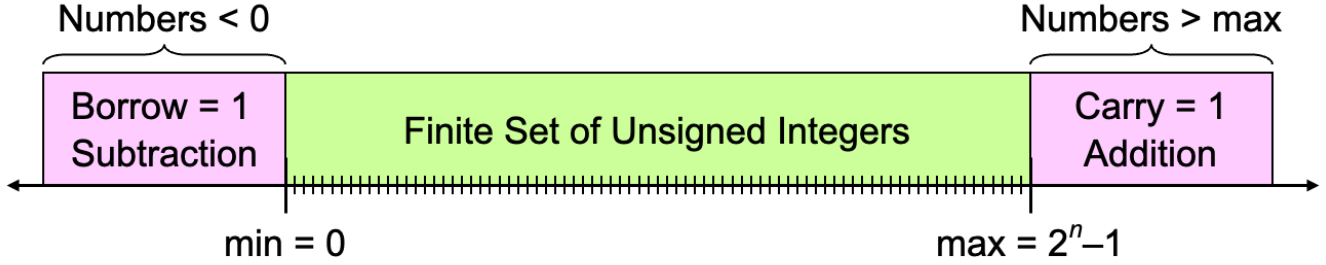



- The "Finite Set" is the range of numbers.
- For n bits, it can represent numbers from 0 to $2^n$ - 1.
  - For example (4-bit): The range is 0 (0000) to 15 (1111).

# Range, Carry, Borrow, and Overflow

## Unit 6: Signed Integer, Logical Operations

- n-bit Unsigned Integers: n-bit representation




- "Carry = 1 for Addition" happens when adding 1 to the largest possible number.
  - MAX_VALUE + 1 "wraps around" to 0.
- In binary addition, this operation generates a Carry-out bit of 1 from the most significant bit (MSB).
- The result in the n-bit register is 0.
  - For example (4-bit): 15 (1111) + 1 = 16 (10000).
    - The 4-bit register can only hold 0000, so the result is 0.
    - The 1 is the carry flag.

```
        0b1111  +
        0b0001
       -------
       0b10000
carry 1 0b0000
```

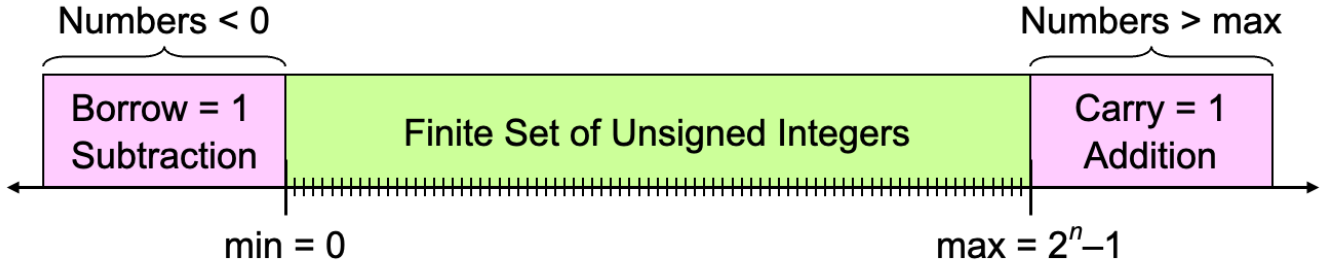
# Range, Carry, Borrow, and Overflow

## Unit 6: Signed Integer, Logical Operations

```
  0b0000
- 0b0001
-------
  0b10000
- 0b0001
-------
  0b1111
```

borrow 1

- n-bit Unsigned Integers: n-bit representation

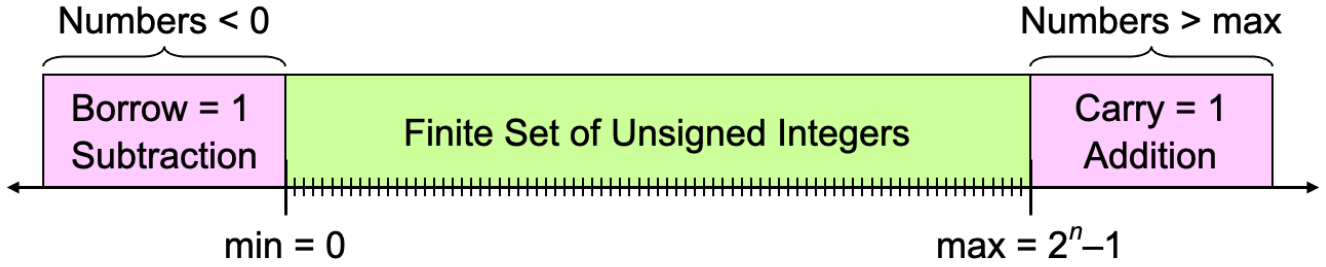



- "Borrow = 1 for Subtraction" happens when subtracting 1 from the smallest possible number (0).
  - 0 - 1 "wraps around" to MAX_VALUE.
- In binary subtraction, this operation requires a Borrow bit of 1.
- The result is the maximum value.
  - For example (4-bit): 0 (0000) - 1.
    - Think of it as borrowing, making it 16 (10000) - 1 = 15 (1111) in the 4-bit register.
    - The result is 15.

# Range, Carry, Borrow, and Overflow

## Unit 6: Signed Integer, Logical Operations

- n-bit Unsigned Integers: n-bit representation



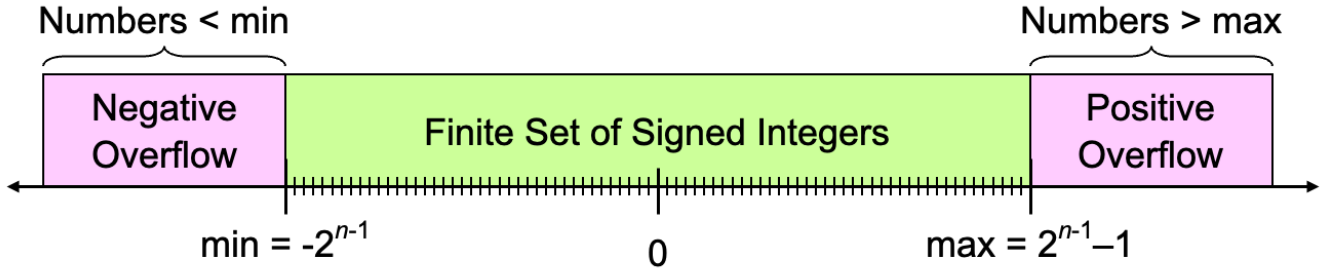

  - The number line is a circle.
  - Counting up past the maximum wraps to the minimum.
  - Counting down past the minimum wraps to the maximum.
  - This is often called an unsigned overflow or wrap-around.

# Integer Signed ADD/SUB Operations

# Range, Carry, Borrow, and Overflow

## Unit 6: Signed Integer, Logical Operations

- Signed Integers: n-bit 2's complement representation

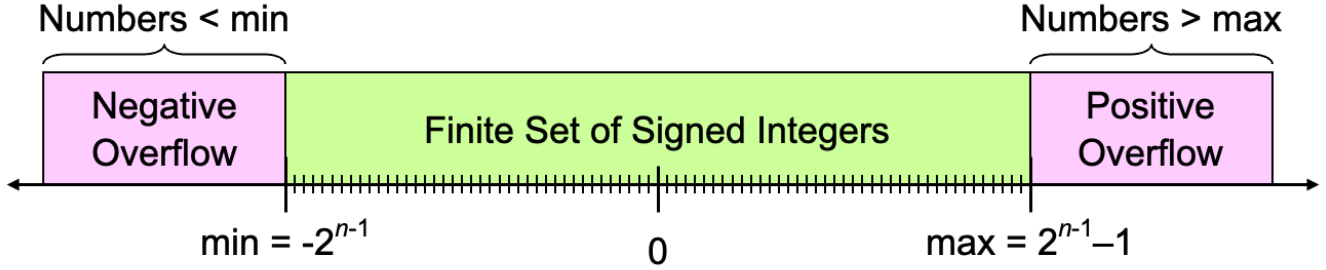



- The "Finite Set" is the range of numbers.
- For n bits, it can represent numbers from $-2^{n-1}$ to $2^{n-1} - 1$.
  - For example (4-bit): The range is -8 (1000) to 7 (0111).

# Range, Carry, Borrow, and Overflow

## Unit 6: Signed Integer, Logical Operations

- Signed Integers: n-bit 2's complement representation




- "Positive Overflow" happens when adding a positive number to the largest positive number, pushing it past the maximum (wraps to the minimum).
- MAX_POSITIVE + 1 becomes the MOST_NEGATIVE number.
- Adding to 0111...1 (max positive) flips the bits to 1000...0 (the most negative number), and the sign bit changes incorrectly.
  - For example (4-bit): 7 (0111) + 1 = 8.
    - But 8 is 1000 in binary, which is -8 in 2's complement.
    - The result is -8, not 8.

```
+0b0111   7
+0b0001 + 1
---------
+0b1000   8   positive overflow
---------
−0b1000  −8   convert to -8
```

# Range, Carry, Borrow, and Overflow

## Unit 6: Signed Integer, Logical Operations

- In binary representation, particularly in the two's complement system, which is the standard for representing signed integers in computers.

- Two's Complement Basics:
  - In two's complement, the most significant bit (MSB) is the sign bit:
    - 0 = positive number
    - 1 = negative number
  - To represent a negative number, we:
    - Start with the positive binary representation.
    - Invert all the bits (this is the one's complement).
    - Add 1 to the result (to get the two's complement).

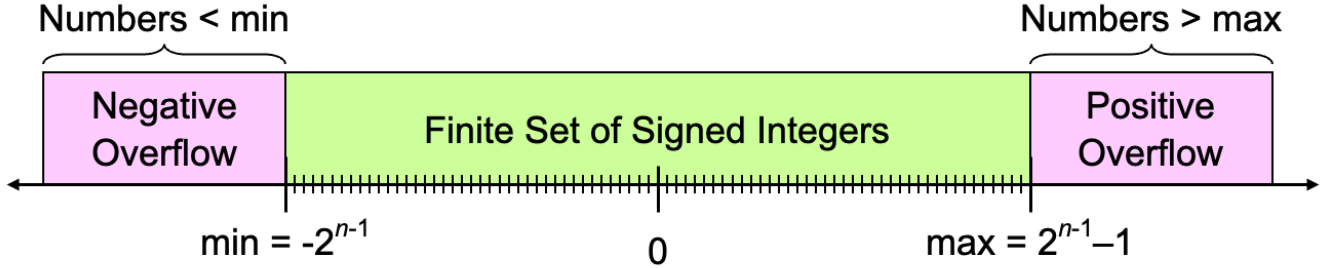
For example: -1

```
0b0001
0b1110 +1
0b1111
```

# Range, Carry, Borrow, and Overflow

## Unit 6: Signed Integer, Logical Operations

- Signed Integers: n-bit 2's complement representation




For example: -1

```
 0b0001
 0b1110 +1
 0b1111
```

- "Negative Overflow" happens when subtracting from the most negative number, pushing it past the minimum (wraps to the maximum).
- MOST_NEGATIVE - 1 becomes the MAX_POSITIVE number.
- Subtracting from 1000...0 (most negative) flips the bits to 0111...1 (the max positive number), and the sign bit changes incorrectly.
  - For example (4-bit):* -8 (1000) - 1 = -9.
    - But -9 would be 10111 in more bits.
    - In 4 bits, we get 0111 (7) because the calculation wraps around.

```
 -0b1000  -8
+-0b1111  -1
 ---------
 -0b10111 -9
```

negative overflow

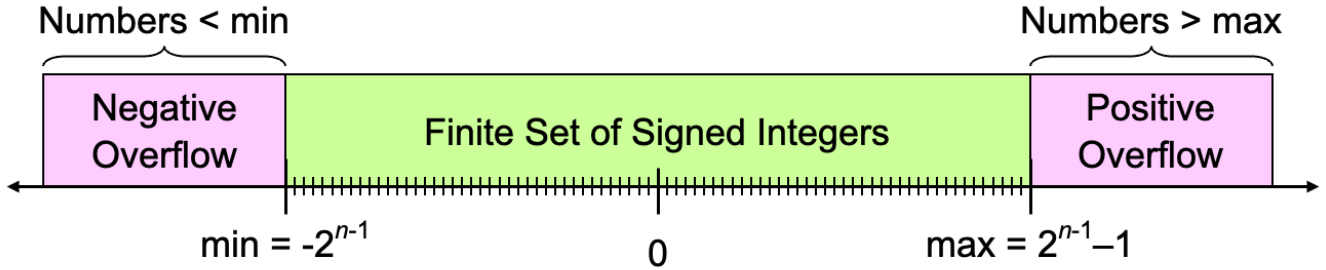
convert to 7

```
 0b0111   7
```

# Range, Carry, Borrow, and Overflow

## Unit 6: Signed Integer, Logical Operations

- Signed Integers: n-bit 2's complement representation




- The number line is also a circle, but the "join" or wrap-around point is between the most positive and most negative numbers, not at zero.
- Overflow is defined based on whether the sign bit changed incorrectly.
  - Positive Overflow: A positive number became negative.
  - Negative Overflow: A negative number became positive.

# Carry & Overflow

## Unit 6: Signed Integer, Logical Operations

- Carry is useful when adding (subtracting) unsigned integers.
  - Carry indicates that the unsigned sum is out of range.
- Overflow is useful when adding (subtracting) signed integers
  - Overflow indicates that the signed sum is out of range.
- Range for 32-bit unsigned integers = 0 to $(2^{32} - 1)$
- Range for 32-bit signed integers = $-2^{31}$ to $(2^{31} - 1)$

# Integer Carry/Overflow Examples

# Carry & Overflow

## Unit 6: Signed Integer, Logical Operations

- For example 1:
  - Carry = 1, Overflow = 0 (NO overflow)
  - Unsigned sum is out of range, but the signed sum is correct

```
        11111 1              1 11     1
A:    1000 0100 0000 0000 1110 0001 0100 0001
B:  + 1111 1111 0000 0000 1111 0101 0010 0000
      ---------------------------------------
Sum:  1000 0011 0000 0001 1101 0110 0110 0001
```

Unsigned sum is out of range, but the signed sum is correct. Both numbers are huge (since the MSB is 1).
A = a large positive number (above 2^{31}
B = a large positive number (above 2^{31}
Their sum will exceed 2^{32} - 1, the max unsigned 32-bit value, so we expect **Carry = 1**.

As Signed Integers (Two's Complement):
MSB=1 means both are negative.
A = a negative number
B = a negative number
Adding two negatives should yield a negative result. Since the result is within the signed range, we expect **Overflow = 0**.

# Carry & Overflow

## Unit 6: Signed Integer, Logical Operations

- For example 2:
  - Carry = 0, Overflow = 1
  - Unsigned sum is correct, but the signed sum is out of range.

```
        01111 1                  11    1
A:    0010 0100 0000 0100 1011 0001 0100 0100
B:  + 0111 1111 0111 0000 0011 0101 0000 0010
      ---------------------------------------
Sum:  1010 0011 0111 0100 1110 0110 0100 0110
```

Let's check the MSB (bit 31) of the operands and the result:

- Number A: MSB = 0 (positive)
- Number B: MSB = 0 (positive)
- Result: MSB = 1 (negative)

Since both operands are positive but the result is negative, this indicates an overflow. Therefore, Overflow = 1.

# Carry, Borrow, Overflow, and Underflow (+ Zero)

# Definitions

## Unit 6: Signed Integer, Logical Operations

4-bit System

- Carry: Occurs during unsigned addition when the result exceeds the maximum value, e.g., >15 in a 4-bit.
    - It's the bit carried out from the Most Significant Bit (MSB).
- Borrow: Occurs during unsigned subtraction when the result would be negative, e.g., <0 in a 4-bit.
    - It indicates a borrow was needed from a higher bit.

# Definitions

## Unit 6: Signed Integer, Logical Operations

4-bit System

- Overflow: Occurs during signed addition or subtraction when the result is outside the representable range, e.g., not between -8 and 7 in a 4-bit.
    - It occurs when two positives add to a negative, two negatives add to a positive, and so on.
- Underflow: This term is often used interchangeably with Borrow in the context of unsigned integers (meaning the result went below zero).
    - In a broader sense, it can also describe a signed result that is too negative (a type of overflow), but it's most clearly understood as the unsigned counterpart to Carry.

# Carry

## Unit 6: Signed Integer, Logical Operations

4-bit System

- Carry: Occurs during unsigned addition when the result exceeds the maximum value, e.g., >15 in a 4-bit.
  - It's the bit carried out from the Most Significant Bit (MSB).
  - Evidence of Carry (Unsigned Addition Overflow):
    - Operation: 15 + 1 (Unsigned)

      Calculation: 1111 + 0001 = (1)0000
    - What happens: The true result is 16, but we only have 4 bits — the answer wraps around to 0.
    - The Evidence: The Carry bit out of the MSB is 1.
  - Analogy:
    - Your car's odometer rolls over from 99999 to 00000.
    - Year in computer today 0000-9999.

# Borrow

## Unit 6: Signed Integer, Logical Operations

4-bit System

- Borrow: Occurs during unsigned subtraction when the result would be negative, e.g., <0 in a 4-bit.
  - It indicates a borrow was needed from a higher bit.
  - Evidence of Borrow/Underflow (Unsigned Subtraction Underflow)
    - Operation: 3 - 5 (Unsigned)

      Calculation: 3 + (-5) = 0011 + 1011 = 1110
    - What happens: The true result is -2, but unsigned numbers can't be negative — the answer wraps around to 14.
    - The Evidence: The Borrow flag is set to 1, indicating that a borrow was required from a non-existent higher bit, which is indicative of an Underflow.
  - Analogy: Trying to take $5 from a wallet that only has $3, you need to borrow $2 from someone else (the higher bit).

# Overflow

## Unit 6: Signed Integer, Logical Operations

4-bit System

- Overflow: Occurs during signed addition or subtraction when the result is outside the representable range (e.g., not between -8 and 7 in a 4-bit).
  - Evidence of OVERFLOW (Signed Addition)
    - Operation: 5 + 5 (Signed)

      Calculation: 0101 + 0101 = 1010
    - What Happens: We added two positive numbers (5 and 5).
    - The result should be positive (10); however, the result 1010 has an MSB of 1, which means negative (-6 in signed). (This is nonsense.)
    - The Evidence: The Overflow flag is set (1) — the rule “positive + positive = negative” is always an error in signed math.
  - Analogy: Adding two large positive numbers and getting a negative number on your calculator.

# Underflow

## Unit 6: Signed Integer, Logical Operations

4-bit System

- Underflow: This term is often used interchangeably with Borrow in the context of unsigned integers (meaning the result went below zero).
  - Evidence of OVERFLOW (Signed Subtraction)
    - Operation: -8 - 1 (Signed)

      Calculation: -8 + (-1) = 1000 + 1111 = (1)0111
    - What Happens: The true result is -9, which is outside the 4-bit signed range — the 4-bit result is 0111 (+7).
    - The Evidence: The Overflow flag is set (1) — the rule “negative + negative = positive” is always an error in signed math.
  - Analogy:
    - It's like pouring so much cold water (negative) into a glass that it spontaneously becomes boiling hot (positive) — it violates the “rules” of the system.

# Carry, Borrow, Overflow, and Underflow

## Unit 6: Signed Integer, Logical Operations

For –2 —> 2 reverse

0b1110
0b0001 +1
0b0010

4-bit System

| Operation | Bit Calculation | Unsigned View & Flags | Signed View & Flags | Evidence for … |
|---|---|---|---|---|
| 15 + 1 | 1111 + 0001 = (1)0000 | 15 + 1 = 16 —> 0<br>Carry = 1<br>Zero = 1 | –1 + 1 = 0 —> 0<br>Overflow = 0<br>Zero = 1 | Carry |
| 3 – 5 | 0011 + 1011 = 1110 | 3 – 5 = –2 —> 14<br>Borrow = 1 (Underflow) | 3 – 5 = –2 —> –2<br>Overflow = 0 | Borrow / Underflow |
| 5 + 5 | 0101 + 0101 = 1010 | 5 + 5 = 10 —> 10<br>Carry = 0 | 5 + 5 = 10 —> –6<br>Overflow = 1 | Overflow |
| –8 – 1 | 1000 + 1111 = (1)0111 | 8 + 15 = 23 —> 7<br>Borrow = 0 Carry = 1 | –8 – 1 = –9 —> 7<br>Overflow = 1 | Overflow |

# Carry, Borrow, Overflow, and Underflow

## Unit 6: Signed Integer, Logical Operations

4-bit System

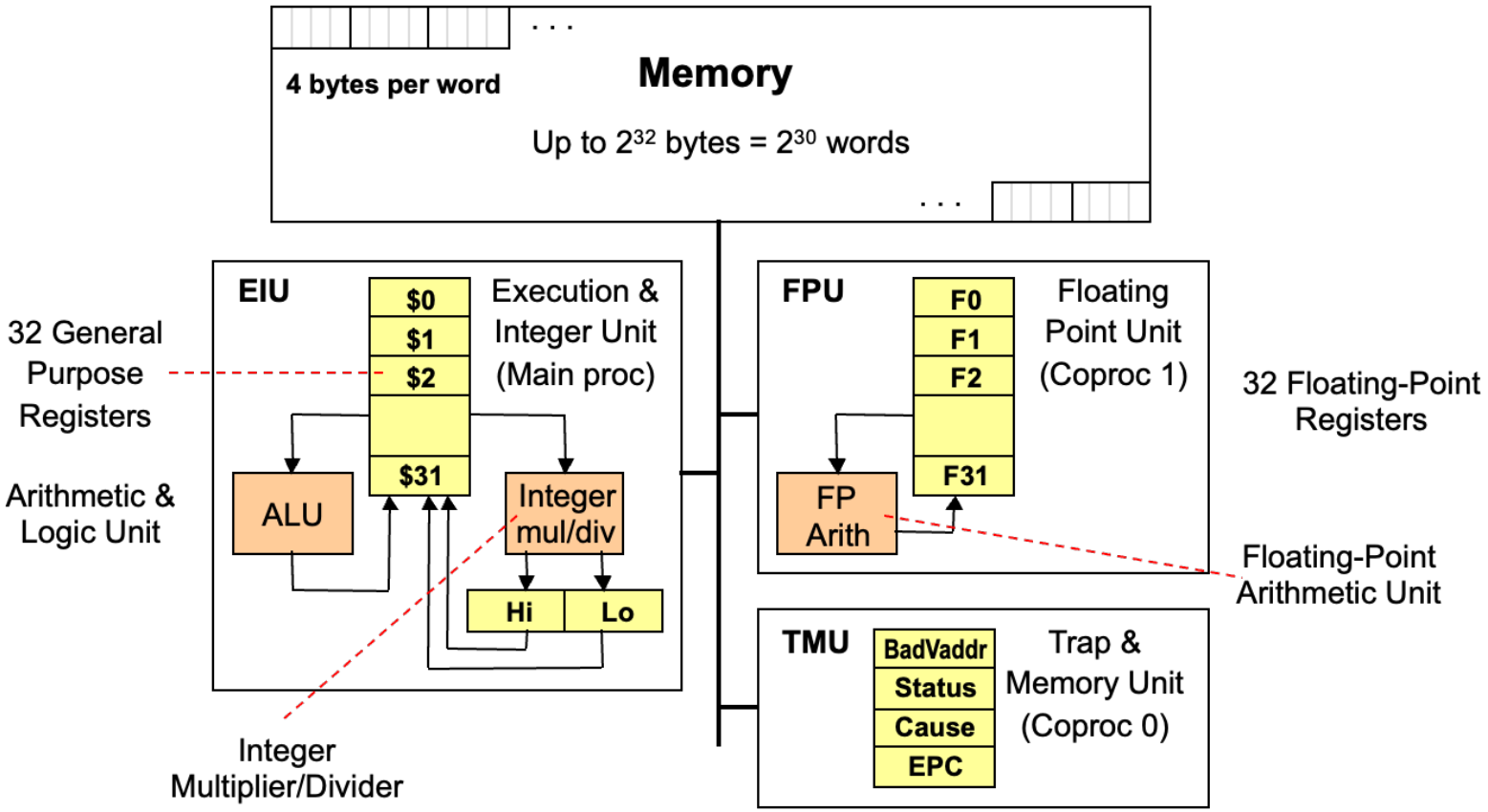
| Operation | Bit Calculation | Unsigned View & Flags | Signed View & Flags | Evidence for … |
|---|---|---|---|---|
| 7 + 7 | | | | |
| 10 – 3 | | | | |
| –4 + 2 | | | | |
| 1 – 1 | | | | |
| 0 + 15 | | | | |
| 7 – 8 | | | | |



# MIPS General-Purpose Registers

# MIPS General-Purpose Registers

## Unit 6: Signed Integer, Logical Operations



# MIPS General-Purpose Registers

## Unit 6: Signed Integer, Logical Operations

- 32 General Purpose Registers (GPRs)
  - An assembler uses the dollar notation to name registers
    - $0 is register 0, $1 is register 1, …, and $31 is register 31
  - All registers are 32-bit wide in MIPS32
  - Register $0 is always zero
    - Any value written to $0 is discarded
- Software conventions
  - Software defines names for all registers
    - To standardize their use in programs
  - Example: $8 - $15 are called $t0 - $t7
    - Used for temporary values

| | |
|---|---|
| $0 = $zero | $16 = $s0 |
| $1 = $at | $17 = $s1 |
| $2 = $v0 | $18 = $s2 |
| $3 = $v1 | $19 = $s3 |
| $4 = $a0 | $20 = $s4 |
| $5 = $a1 | $21 = $s5 |
| $6 = $a2 | $22 = $s6 |
| $7 = $a3 | $23 = $s7 |
| $8 = $t0 | $24 = $t8 |
| $9 = $t1 | $25 = $t9 |
| $10 = $t2 | $26 = $k0 |
| $11 = $t3 | $27 = $k1 |
| $12 = $t4 | $28 = $gp |
| $13 = $t5 | $29 = $sp |
| $14 = $t6 | $30 = $fp |
| $15 = $t7 | $31 = $ra |

# MIPS Register Conventions

## Unit 6: Signed Integer, Logical Operations

- An assembler can refer to registers by name or by number.
- It is easier for you to remember registers by name.
- An assembler converts a register name to its corresponding number.

| Name | Register | Usage |
|---|---|---|
| $zero | $0 | Always 0 (forced by hardware) |
| $at | $1 | Reserved for assembler use |
| $v0 – $v1 | $2 – $3 | Result values of a function |
| $a0 – $a3 | $4 – $7 | Arguments of a function |
| $t0 – $t7 | $8 – $15 | Temporary Values |
| $s0 – $s7 | $16 – $23 | Saved registers (preserved across call) |
| $t8 – $t9 | $24 – $25 | More temporaries |
| $k0 – $k1 | $26 – $27 | Reserved for OS kernel |
| $gp | $28 | Global pointer (points to global data) |
| $sp | $29 | Stack pointer (points to top of stack) |
| $fp | $30 | Frame pointer (points to stack frame) |
| $ra | $31 | Return address (used by jal for function call) |

# Instruction Formats

## Unit 6: Signed Integer, Logical Operations

- All instructions are 32-bit wide. Three instruction formats:
  - Register (R-Type)
    - Register-to-register instructions.
    - Op: operation code specifies the format of the instruction.




  - Immediate (I-Type)
    - A 16-bit immediate constant is part of the instruction.




  - Jump (J-Type)
    - Used by jump instructions.



# R-Type Operation Example

## Unit 6: Signed Integer, Logical Operations

- Consider the translation of: `f = (g+h)-(I+j).`
- A programmer/compiler allocates registers to variables
- Given that: `$t0=f`, `$t1=g`, `$t2=h`, `$t3=i`, and `$t4=j`
- Called temporary registers: `$t0=$8`, `$t1=$9`, …
- Translation of: `f = (g+h)-(i+j)`

```
addu  $t5, $t1, $t2  # $t5 = g + h
addu  $t6, $t3, $t4  # $t6 = i + j
subu  $t0, $t5, $t6  # f = (g+h)-(i+j)
```

- Assembler translates `addu $t5, $t1, $t2` into binary code.

| Op | $t1 | $t2 | $t5 | sa | addu |
|---|---|---|---|---|---|
| 000000 | 01001 | 01010 | 01101 | 00000 | 100001 |

# MIPS Special Opcode Encoding of Function

## Unit 6: Signed Integer, Logical Operations

| function | | bits 2..0 | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| bits 5. 3 | | 000 | 001 | 010 | 011 | 100 | 101 | 110 | 111 |
| 0 | 000 | SLL[1] | *MOVCI* [6R]δ | *SRL* δ | SRA | SLLV | LSA[6N] | *SRLV* δ | SRAV |
| 1 | 001 | JR[2,3,6R] | JALR[2] | MOVZ[6R] | MOVN[6R] | SYSCALL | BREAK | SDBBP[6Nm] | SYNC |
| 2 | 010 | MFHI[6R]<br>CLZ[6Nm] | MTHI[6R]<br>CLO[6Nm] | MFLO[6R] | MTLO[6R] | β | β | β | β |
| 3 | 011[4] | [4]MULT[6R]<br>SOP30[6N] | [4]MULTU[6R]<br>SOP31[6N] | [4]DIV[6R]<br>SOP32[6N] | [4]DIVU[6R]<br>SOP33[6N] | β | β | β | β |
| 4 | 100 | ADD | ADDU | SUB | SUBU | AND | OR | XOR | NOR |
| 5 | 101 | * | * | SLT | SLTU | β | β | β | β |
| 6 | 110 | TGE | TGEU | TLT | TLTU | TEQ | SELEQZ[6N] | TNE | SELNEZ[6N] |
| 7 | 111 | β | * | β | β | β | * | β | β |

1. Specific encodings of the *rt*, *rd*, and *sa* fields are used to distinguish among the SLL, NOP, SSNOP, EHB and PAUSE functions. Release 6 makes SSNOP equivalent to NOP.
2. Specific encodings of the *hint* field are used to distinguish JR from JR.HB and JALR from JALR.HB

# Assembly Language Statements

## Unit 6: Signed Integer, Logical Operations

- Typically, one statement should appear on a line.
- Three types of statements in assembly language:
  1. Executable Instructions
     - Generate machine code for the processor to execute at runtime.
     - Instructions tell the processor what to do.
  2. Pseudo-Instructions and Macros
     - Translated by the assembler into real instructions.
     - Simplify the programmer's task.
  3. Assembler Directives
     - Provide information to the assembler while translating a program.
     - Used to define segments, allocate memory variables, etc.
     - Non-executable: directives are not part of the instruction set.

# Instructions

## Unit 6: Signed Integer, Logical Operations

- Assembly language instructions have the format:

  `[label:]  mnemonic  [operands]  [#comment]`

  - Label: (optional)
    - Marks the address of a memory location, must have a colon.
    - Typically appear in data and text segments.
  - Mnemonic
    - Identifies the operation, e.g., add, sub, etc.
  - Operands
    - Specify the data required by the operation
    - Operands can be registers, memory variables, or constants
    - Most instructions have three operands
- L1: addiu $t0, $t0, 1 #increment $t0

# Logic Bitwise Operations

# Logic Bitwise Operations

## Unit 6: Signed Integer, Logical Operations

- Logic bitwise operations: `and`, `or`, `xor`, `nor`

| x | y | x and y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

| x | y | x or y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

| x | y | x xor y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

| x | y | x nor y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |

# Logic Bitwise Operations

## Unit 6: Signed Integer, Logical Operations

- Logic bitwise operations: `and`, `or`, `xor`, `nor`

| x | y | x and y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

| x | y | x or y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

| x | y | x xor y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

| x | y | x nor y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |

- AND instruction is used to clear bits: `x and 0` → `0`
- OR instruction is used to set bits: `x or 1` → `1`
- XOR instruction is used to toggle bits: `x xor 1` → `not x`
- NOT instruction is not needed, why?
  - `not $t1, $t2` is equivalent to: `nor $t1, $t2, $t2`

# The Logic: Understanding NOR

## Unit 6: Signed Integer, Logical Operations

- Logic bitwise operations: `and`, `or`, `xor`, `nor`

| x | y | x and y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

| x | y | x or y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

| x | y | x xor y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

| x | y | x nor y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |

- Now, what happens if we make both inputs the same?
- Let's set A = X and B = X.
- The truth table simplifies to:
  - Because 0 `nor` 0 is 1 → `not` 0 is 1
  - Because 1 `nor` 1 is 0) → `not` 1 is 0
  - Therefore, `x nor x` is logically identical to `not x`.

| x | x | x nor x |
|---|---|---|
| 0 | 0 | 1 |
| | | |
| | | |
| 1 | 1 | 0 |

# Logic Bitwise Instructions

## Unit 6: Signed Integer, Logical Operations

| Instruction | Meaning | Op | Rs | Rt | Rd | sa | func |
|---|---|---|---|---|---|---|---|
| `and $t1, $t2, $t3` | `$t1 = $t2 & $t3` | 0 | $t2 | $t3 | $t1 | 0 | 0x24 |
| `or $t1, $t2, $t3` | `$t1 = $t2 \| $t3` | 0 | $t2 | $t3 | $t1 | 0 | 0x25 |
| `xor $t1, $t2, $t3` | `$t1 = $t2 ^ $t3` | 0 | $t2 | $t3 | $t1 | 0 | 0x26 |
| `nor $t1, $t2, $t3` | `$t1 = ~($t2 \| $t3)` | 0 | $t2 | $t3 | $t1 | 0 | 0x27 |

Examples:

- Given: `$t1` = `0xabcd1234` and `$t2` = `0xffff0000`

```
and $t0, $t1, $t2        # $t0 = 0x abcd0000
```

- This performs a bitwise `AND` between `$t1` and `$t2`, storing the result in `$t0`.

```
# $t1 = 0xabcd1234 → Binary: 1010 1011 1100 1101 0001 0010 0011 0100
# $t2 = 0xffff0000 → Binary: 1111 1111 1111 1111 0000 0000 0000 0000
# $t0 = 0x abcd0000 → Binary: 1010 1011 1100 1101 0000 0000 0000 0000
```

# Logic Bitwise Instructions

## Unit 6: Signed Integer, Logical Operations

| Instruction | Meaning | Op | Rs | Rt | Rd | sa | func |
|---|---|---|---|---|---|---|---|
| `and $t1, $t2, $t3` | `$t1 = $t2 & $t3` | 0 | $t2 | $t3 | $t1 | 0 | 0x24 |
| `or $t1, $t2, $t3` | `$t1 = $t2 \| $t3` | 0 | $t2 | $t3 | $t1 | 0 | 0x25 |
| `xor $t1, $t2, $t3` | `$t1 = $t2 ^ $t3` | 0 | $t2 | $t3 | $t1 | 0 | 0x26 |
| `nor $t1, $t2, $t3` | `$t1 = ~($t2 \| $t3)` | 0 | $t2 | $t3 | $t1 | 0 | 0x27 |

Examples:

- Given: $t1 = 0xabcd1234 and $t2 = 0xffff0000

```
and $t0, $t1, $t2        # $t0 = 0xabcd0000
or  $t0, $t1, $t2        # $t0 = 0xffff1234
xor $t0, $t1, $t2        # $t0 = 0x54321234
nor $t0, $t1, $t2        # $t0 = 0x0000edcb
```



# Logic Bitwise Instructions

## Unit 6: Signed Integer, Logical Operations

| Instruction | Meaning | Op | Rs | Rt | Rd | sa | func |
|---|---|---|---|---|---|---|---|
| `and $t1, $t2, $t3` | `$t1 = $t2 & $t3` | 0 | $t2 | $t3 | $t1 | 0 | 0x24 |
| `or $t1, $t2, $t3` | `$t1 = $t2 \| $t3` | 0 | $t2 | $t3 | $t1 | 0 | 0x25 |
| `xor $t1, $t2, $t3` | `$t1 = $t2 ^ $t3` | 0 | $t2 | $t3 | $t1 | 0 | 0x26 |
| `nor $t1, $t2, $t3` | `$t1 = ~($t2 \| $t3)` | 0 | $t2 | $t3 | $t1 | 0 | 0x27 |

Examples:

- Given: $t1 = 0xabcdf214 and $t2 = 0xffff0201

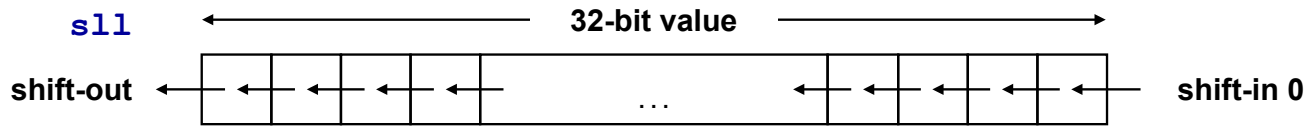
```
and $t0, $t1, $t2      # $t0 = 0xabcd0200
or  $t0, $t1, $t2      # $t0 = 0xfffff215
xor $t0, $t1, $t2      # $t0 = 0x5432f015
nor $t0, $t1, $t2      # $t0 = 0x00000dea
```



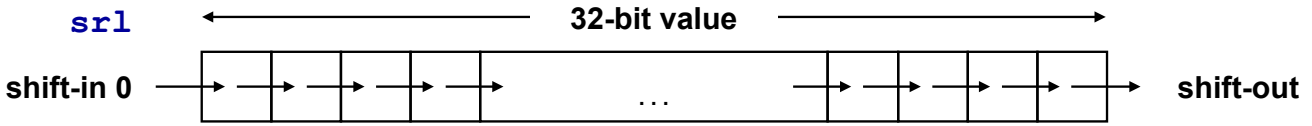
# Shift Operations

## Unit 6: Signed Integer, Logical Operations

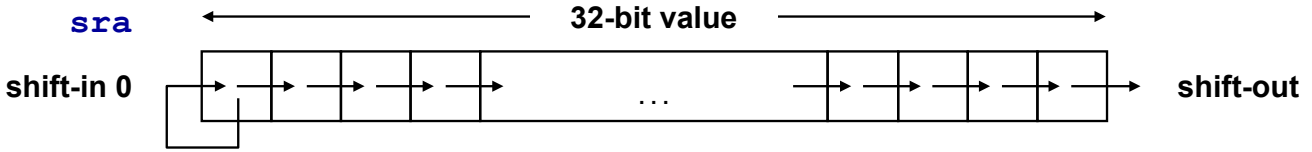
- Shifting is the process of moving the 32 bits of a number left or right.
- These instructions use the 5-bit shift amount field.
- **`sll`** means shift left logical (insert zero from the right).




- **`srl`** means shift right logical (insert zero from the left).




- **`sra`** means shift right arithmetic (insert sign-bit).



# Shift Instructions

## Unit 6: Signed Integer, Logical Operations

| Instruction | Meaning | Op | Rs | Rt | Rd | sa | func |
|---|---|---|---|---|---|---|---|
| `sll $t1,$t2,10` | `$t1 = $t2 <<< 10` | 0 | 0 | $t2 | $t1 | 10 | 0x0 |
| `srl $t1,$t2,10` | `$t1 = $t2 >>> 10` | 0 | 0 | $t2 | $t1 | 10 | 0x2 |
| `sra $t1,$t2,10` | `$t1 = $t2 >>> 10` | 0 | 0 | $t2 | $t1 | 10 | 0x3 |
| `sllv $t1,$t2,$t3` | `$t1 = $t2 <<< $t3` | 0 | $t3 | $t2 | $t1 | 0 | 0x4 |
| `srlv $t1,$t2,$t3` | `$t1 = $t2 >>> $t3` | 0 | $t3 | $t2 | $t1 | 0 | 0x6 |
| `srav $t1,$t2,$t3` | `$t1 = $t2 >>> $t3` | 0 | $t3 | $t2 | $t1 | 0 | 0x7 |

- **`sll`**, **`srl`**, **`sra`**: shift by a constant amount.
  - The shift amount (sa) field specifies a number between 0 and 31.
- **`sllv`**, **`srlv`**, **`srav`**: shift by a variable amount.
  - A source register specifies the variable shift amount, ranging from 0 to 31.
  - Only the lower 5 bits of the source register are used as the shift amount.

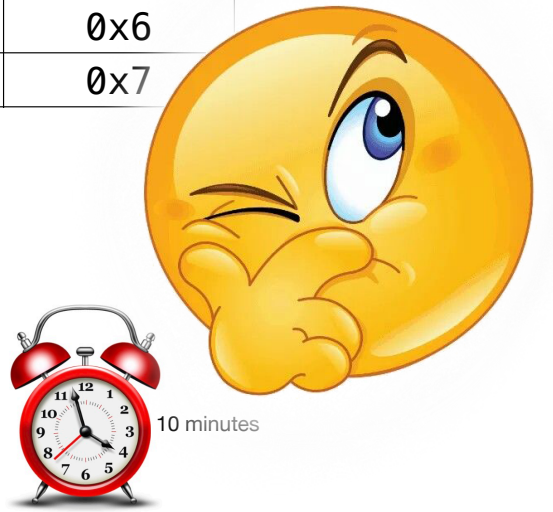
# Shift Instruction Examples

## Unit 6: Signed Integer, Logical Operations

| Instruction | Meaning | Op | Rs | Rt | Rd | sa | func |
|---|---|---|---|---|---|---|---|
| sll $t1,$t2,10 | $t1 = $t2 <<< 10 | 0 | 0 | $t2 | $t1 | 10 | 0x0 |
| srl $t1,$t2,10 | $t1 = $t2 >>> 10 | 0 | 0 | $t2 | $t1 | 10 | 0x2 |
| sra $t1,$t2,10 | $t1 = $t2 >>> 10 | 0 | 0 | $t2 | $t1 | 10 | 0x3 |
| sllv $t1,$t2,$t3 | $t1 = $t2 <<< $t3 | 0 | $t3 | $t2 | $t1 | 0 | 0x4 |
| srlv $t1,$t2,$t3 | $t1 = $t2 >>> $t3 | 0 | $t3 | $t2 | $t1 | 0 | 0x6 |
| srav $t1,$t2,$t3 | $t1 = $t2 >>> $t3 | 0 | $t3 | $t2 | $t1 | 0 | 0x7 |

- Given that: $t2 = 0xabcd1234 and $t3 = 16

```
sll  $t1, $t2, 8        # $t1 = 0xcd123400
srl  $t1, $t2, 4        # $t1 = 0x0abcd123
sra  $t1, $t2, 4        # $t1 = 0xfabcd123
srlv $t1, $t2, $t3      # $t1 = 0x0000abcd
```



# References

# References

## Reference Materials


- William Stallings, “Computer Organization and Architecture Designing for Performance,” Tenth Edition, *Pearson*, 2016
- Aiman El-Maleh, “Computer Organization: Lectures,” https://faculty.kfupm.edu.sa, accessed August 26, 2025

